

***MORE* FUNCTIONAL PROGRAMMING IN PYTHON**

FILTER

`filter` is a higher order function that takes in a function and sequence and returns a new sequence containing only those elements for which the predicate function evaluates to `True`.

`filter(f, seq)` is equivalent to:

```
[elem for elem in seq if f(elem)]
```

```
res = []
for elem in seq:
    if f(elem):
        res.append(elem)
```

```
def find_posts(posts, user):  
    output = []  
    for post in posts:  
        if post.user == user:  
            output.append(post)  
    return output
```

```
def find_evens(numbers):  
    output = []  
    for num in numbers:  
        if num % 2 == 0:  
            output.append(num)  
    return output
```

```
def find_long_words(words, min_length):  
    output = []  
    for word in words:  
        if len(word) >= min_length:  
            output.append(word)  
    return output
```

```
def find_passing_grades(grades):  
    output = []  
    for grade in grades:  
        if grade >= 60:  
            output.append(grade)  
    return output
```

It's all **filter**. All of it.

If `numbers = [4, 9, 10, 13, 18, 2, 0]`

```
output = []  
for num in numbers:  
    if num % 2 == 0:  
        output.append(num)  
print(output)
```

```
def is_even(num):  
    return num % 2 == 0  
output = list(filter(is_even, numbers))  
print(output)
```

both examples print `[4, 10, 18, 2, 0]`

If `words = ["time", "cast", "a", "spell", "on", "you"]`

```
output = []
for word in words:
    if len(word) >= 4:
        output.append(word)
print(output)
```

```
def is_long(word):
    _____
output = list(filter(is_long, words))
print(output)
```

L11 What goes in the blank so that both snippets print `["time", "cast", "spell"]`?

MAP

`map` is a higher order function that takes in a function and sequence and returns a new sequence containing elements of the input sequence after having `f` applied to them.

`map(f, seq)` is equivalent to:

```
[f(elem) for elem in seq]
```

```
res = []  
for elem in seq:  
    res.append(f(elem))
```

Help! What are some ways that we could curve these scores by mapping?

```
exam_scores = [90, 89, 41, 100, 82, 71, 66]
```

PRACTICE! WHAT DO EACH OF THESE EVALUATE TO?

S7, S8, S9 If `names = ["travis", "Harry", "talie", "adi", "Sukya", "Molly"]`, what's the result?

```
def perfect_name(name):  
    return len(name) == 5 and 'a' in name  
result = list(filter(perfect_name, names))
```

```
def f(n):  
    return n[0] + n[-1]  
result = list(map(f, names))
```

```
l = filter(perfect_name, names)  
l = filter(str.islower, l)  
result = list(map(str.upper, l))
```


PRACTICE! IDENTIFYING THE PATTERN

Is this task a map, a filter, or both?

- get a list of all the capital letters in a string
- given a list of numbers, get another list of numbers that has the squares of those numbers
- given a list of numbers, get another list of numbers that has the squares of those numbers that are less than 25

TOGETHER, ALOUD, IN CONCERT

Without `map/filter`: Given a list of numbers, print another list of numbers that has the squares of those numbers that are less than 25. (the squares, not the original numbers)

```
numbers = [3, -6, 8, 4, 4.1]
```

```
output = []
```

```
print(output)
```

L11

Using `map/filter`: Given a list of numbers, print another list of numbers that has the squares of those numbers that are less than 25. (the squares, not the original numbers) *Use the helper functions defined here.*

```
def small_enough(n):  
    return n < 25  
def square(n):  
    return n * n  
numbers = [3, -6, 8, 4, 4.1]  
# TODO  
# TODO  
print(output)
```

  We are warmed up now, yes?  

Very good, on to the new stuff.

LAMBDA

Lambdas (or anonymous functions) are functions that are defined without names.

```
lambda <parameter_list>: expression
```

would become

```
def no_name(<parameter_list>):  
    return expression
```

(but not LITERALLY `no_name` because the point is that the function object does not actually have a name.)

Some fresh, clean lambdas for you to consider:

```
def plus_two(n):  
    return n + 2
```

```
lambda n: n + 2
```

```
#####
```

```
def email_from_name(first, last):  
    return last[0] + first + "@seas.upenn.edu"
```

```
lambda first, last: last[0] + first + "@seas.upenn.edu"
```

```
#####
```

```
def name_in_list_and_long(name, l):  
    in_list = name in l  
    is_long = len(name) >= 5  
    return in_list and is_long
```

```
lambda name, l: name in l and len(name) >= 5
```

For your own individual perusal: an example that **can** be turned into a lambda despite not being a typical example of how you would use a lambda.

```
def pass_fail(grade):  
    if grade > 62:  
        return "Pass"  
    return "Fail"
```

```
lambda grade: "Pass" if grade > 62 else "Fail" # requires some new syntax
```

APPLYING HIGHER ORDER FUNCTIONS TO PAST PROBLEMS

Remember the `caesar` homework?

We wrote a function called `string_to_symbol_list()` that took a string and returned a list but all characters were converted to symbols. `ord(charater) - 65`

What does this sound like?

Try implementing it with a higher order function and lambda that represents the shifting logic. **L13**

```
def string_to_symbol_list(string):  
    # TODO
```


REDUCE

Aggregation is done using a slightly trickier HOF called `reduce` imported from `functools`.

`reduce` is a function that takes in an *aggregation function* and a sequence and repeatedly accumulates elements from the sequence using the aggregation function.

`reduce(f, seq)` is roughly equivalent to:

```
result = seq[0]
for elem in seq[1:]:
    result = f(result, elem)
```

CUMULATIVE PRODUCT

```
seq = [4, 3, 2, 1]
result = seq[0]
for elem in seq[1:]:
    # loop start
    result = result * elem
    # loop end
```

ELEM	RESULT AT START	RESULT AT THE END
3	4	product(4, 3) -> 12
2	12	product(12, 2) -> 24
1	24	product(24, 1) ->24

CUMULATIVE PRODUCT

```
def product(accumulator, update):  
    return accumulator * update  
  
seq = [4, 3, 2, 1]  
result = seq[0]  
for elem in seq[1:]:  
    # loop start  
    result = product(result, elem)  
    # loop end
```

ELEM	RESULT AT START	RESULT AT THE END
3	4	product(4, 3) -> 12
2	12	product(12, 2) -> 24
1	24	product(24, 1) -> 24

Same idea, but think of the multiplication as a call to an aggregating function.

REDUCE, PROPERLY

```
def product(accumulator, update):  
    return accumulator * update  
output = reduce(product, [4, 3, 2, 1])
```

product(product(product(4, 3), 2), 1)

product(product(12, 2), 1)

product(24, 1)

24

What's the value of output? Can you think of a better name for the function `s7`?

```
def s10(accumulator, update):  
    return update and accumulator  
output = reduce(s10, [True, True, False, True])
```

Hint: the above is equivalent to:

```
seq = [True, True, False, True]  
result = seq[0]  
for elem in seq[1:]:  
    result = s10(result, elem)
```

```
def s10(accumulator, update):  
    return update and accumulator
```

```
seq = [True, True, False, True]  
result = seq[0]  
for elem in seq[1:]:  
    # loop start  
    result = s10(result, elem)  
    # loop end
```

ELEM	RESULT AT START	RESULT AT THE END
True	True	s10(True, True) -> True
False	True	s10(True, False) -> False
True	False	s10(False, True) -> False ✓

S8 What's the value of output? Can you think of a better name for the function `s8`?

```
def s8(accumulator, update):  
    return update or accumulator  
output = reduce(s8, [True, True, False, True])
```

PRACTICE CODE WRITING!

Given a list of characters, take only the ones which are singular lowercase letters and mash them up into character*3 in a string. e.g. `["I", "fear", "a", "fall"]` becomes `"IIIaaa"`.

(Needs all three HOFs at this point.)

C14

APPLYING HOF TO CAESAR

```
def shift(symbol, n):  
    if 0 <= symbol <= 25:  
        return (symbol + n) % 26  
    return symbol
```

assume `n` is the amount we want to shift each symbol by

```
def encrypt(to_encode, n):  
    symbols = string_to_symbol_list(to_encode)  
    # TODO: put something here  
    return symbol_list_to_string(symbols)
```

Can you use two higher order functions, `shift` and perhaps another lambda to reimplement encrypt? **C12**

THIRD ARGUMENT TO REDUCE

The **initial value** can be chosen as something other than the first value in the sequence.

`reduce(f, seq, initial)` is roughly equivalent to:

```
result = initial
for elem in seq:
    result = f(result, elem)
```

```

def prepend(accumulator, update):
    return [update] + accumulator

seq = [1, 2, 3, 4]
result = []           # initial value is not an element of the sequence
for elem in seq:      # start iterating at the first element of the sequence
    # start
    result = prepend(result, elem)
    # end

```

ELEM	RESULT AT START	RESULT AT THE END
1	[]	prepend([], 1) -> [1]
2	[1]	prepend([1], 2) -> [2, 1]
3	[2, 1]	prepend([2, 1], 3) -> [3, 2, 1]
4	[3, 2, 1]	prepend([3, 2, 1], 4) -> [4, 3, 2, 1]

```
reduce(prepend, [1, 2, 3, 4], [])
```

```
prepend(prepend(prepend(prepend([], 1), 2), 3), 4)
```

```
prepend(prepend(prepend([1], 2), 3), 4)
```

```
prepend(prepend([2, 1], 3), 4)
```

```
prepend([3, 2, 1], 4)
```

```
[4, 3, 2, 1]
```

